

CESAR SCHOOL

Documentação Técnica

Dashboard de Meios de Pagamento · Banco Central do Brasil

Lucas Cardoso & André Montenegro

Professor: Marco Aurelio Tomaz Mialaret Junior

Recife, 2026

Sumário

1	Introdução	1
1.1	Motivação	1
1.2	Objetivos	1
2	Arquitetura do Sistema	1
2.1	Visão Geral	1
2.2	Infraestrutura	2
3	Fonte de Dados	2
3.1	API do Banco Central do Brasil	2
3.2	Principais Campos	2
3.2.1	Qualidade dos Dados	3
4	Pipeline de Dados	3
4.1	Camada Bronze — Extração	3
4.2	Camada Silver — Tratamento	3
4.3	Camada Gold — Análise	4
5	Dashboard	4
6	Execução	4
7	Dependências	5
8	Considerações Finais	5

1 • Introdução

Este documento apresenta a documentação técnica do projeto **BCB Dashboard**, desenvolvido como trabalho acadêmico para a disciplina de Engenharia de Dados na Cesar School.

O projeto tem como objetivo coletar, processar e visualizar dados abertos do Banco Central do Brasil (BCB), especificamente os dados de Meios de Pagamento Trimestral, cobrindo o período de 2015 a 2026. O resultado é um painel interativo que permite analisar a evolução do sistema financeiro brasileiro ao longo da última década.

1.1 • Motivação

A transformação digital dos meios de pagamento no Brasil acelerou significativamente a partir de 2020, impulsionada pela pandemia de COVID-19 e pelo lançamento do Pix. Este projeto nasce da necessidade de quantificar e visualizar essas mudanças a partir de dados oficiais do Banco Central.

1.2 • Objetivos

- Extrair dados da API pública do Banco Central do Brasil;
- Aplicar a arquitetura medalhão (Bronze → Silver → Gold) para tratamento dos dados;
- Persistir os dados em banco de dados relacional (PostgreSQL);
- Apresentar análises visuais através de um dashboard interativo em Streamlit.

2 • Arquitetura do Sistema

2.1 • Visão Geral

O sistema segue a arquitetura medalhão (*Medallion Architecture*), que organiza o processamento em camadas progressivas de qualidade.

Camada	Arquivo	Responsabilidade
Bronze	bronze.py	Extração dos dados crus da API do BCB e persistência no banco.
Silver	silver.py	Limpeza, normalização de tipos e ordenação cronológica.
Gold	gold.py	Transformações analíticas e criação de tabelas por análise.
App	app.py	Dashboard interativo com visualizações via Streamlit.

Tabela 1: Camadas da arquitetura medalhão.

2.2 · Infraestrutura

O banco de dados é o **PostgreSQL**, executado em contêiner Docker. A comunicação é feita via **SQLAlchemy**.

```
# docker-compose.yml
services:
  postgres:
    image: postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: bcb_dashboard
    ports:
      - "5432:5432"
```

3 · Fonte de Dados

3.1 · API do Banco Central do Brasil

Os dados são obtidos do endpoint `MeiosdePagamentosTrimestralDA` da API OData do BCB.

https://olinda.bcb.gov.br/olinda/servico/MPV_DadosAbertos/versao/v1/odata/MeiosdePagamentosTrimestralDA

3.2 · Principais Campos

Campo	Tipo	Descrição
datatrimestre	String → DateTime	Data de referência do trimestre.
quantidadeSaques	Float	Volume de saques (em milhares).
valorTED	Float	Valor total de TEDs (em milhões).
valorPix	Float	Transações Pix (disponível a partir de nov/2020).
valorTEC	Float	TEC descontinuada — valores zerados em todo o período.

Tabela 2: Principais campos retornados pela API.

Qualidade dos Dados

- valorTEC está zerado em todo o período: o instrumento foi descontinuado.
- valorPix e quantidadePix são nulos antes do 4.º trimestre de 2020.
- quantidadePix é armazenado como float mas representa valores em milhares.

4 · Pipeline de Dados

4.1 · Camada Bronze — Extração

```
def data_extract():
    url = "https://olinda.bcb.gov.br/.../MeiosdePagamentosTrimestralDA
        (trimestre=@trimestre)?@trimestre='20151'&$top=100&$format=json"
    response = requests.get(url)
    df = pd.DataFrame(response.json()['value'])
    engine = create_engine("postgresql://postgres:postgres@localhost:5432/bcb_dashboard")
    df.to_sql('bronze_meios_pagamento', engine, if_exists='replace', index=False)
    return df
```

4.2 · Camada Silver — Tratamento

```
def silver_data_extract():
    df = data_extract()
    df['datatrimestre'] = pd.to_datetime(df['datatrimestre'])
    df = df.sort_values('datatrimestre').reset_index(drop=True)
    engine = create_engine("postgresql://postgres:postgres@localhost:5432/bcb_dashboard")
    df.to_sql('silver_meios_pagamento', engine, if_exists='replace', index=False)
    return df
```

Transformações: conversão de tipo, ordenação cronológica, reset de índice.

4.3 · Camada Gold — Análise

```
# Variação percentual de saques
gold_saques['variacao_porcentual'] = df['quantidadeSaques'].pct_change() * 100

# Rotulação por período (pré/pós Pix)
gold_ted['periodo'] = gold_ted['datatrimestre'].apply(
    lambda x: 'pos_pix' if x >= pd.to_datetime('2020-11-01') else 'pre_pix'
)
```

Tabelas geradas: gold_volume_total, gold_saques, gold_ted_cres.

5 · Dashboard

O dashboard é construído com **Streamlit** e gráficos via **Matplotlib**. Três análises estão disponíveis em abas separadas:

- **Queda de Saques (2015–2026)** — série temporal com marcadores da pandemia e do Pix;
- **Volume Total de Transações** — crescimento do fluxo financeiro agregado;
- **TED: Crescimento e Estagnação** — inflexão do instrumento após o Pix.

6 · Execução

```
# 1. Ambiente virtual
python3 -m venv venv && source venv/bin/activate

# 2. Dependências
pip install -r requirements.txt

# 3. Banco de dados
```

```
docker compose up -d
```

```
# 4. Pipeline
```

```
python bronze.py && python silver.py && python gold.py
```

```
# 5. Dashboard
```

```
streamlit run app.py # http://localhost:8501
```

7 • Dependências

Biblioteca	Finalidade
streamlit	Interface web do dashboard
pandas	Manipulação e transformação de dados
requests	Requisições HTTP à API do BCB
sqlalchemy	ORM e conexão com PostgreSQL
matplotlib	Geração de gráficos
python-dotenv	Variáveis de ambiente
psycopg2	Driver PostgreSQL para Python

Tabela 3: Dependências do projeto.

8 • Considerações Finais

A arquitetura medalhão garante rastreabilidade e separação de responsabilidades entre as camadas. O uso de Docker e de variáveis de ambiente assegura portabilidade do ambiente. O pipeline é linear e reproduzível: qualquer execução a partir dos dados brutos gera os mesmos resultados analíticos.